The Structure-Preserving Doubling Algorithm for the Algebraic Riccati Equation

Amp

A linear-quadratic regulator needs the stabilizing solution of an algebraic Riccati equation. The classical way to reach it extends the control horizon one step at a time, and it slows down badly when the closed loop sits near the stability boundary. This note explains the alternative that the doubling algorithm is built on: each step of that horizon recursion is one application of a symplectic matrix to the graph of the current cost, so squaring the matrix doubles the horizon. Forming the powers directly destroys the answer, so the algorithm squares a block **factorization** instead, which is the structure-preserving doubling algorithm (SDA). The running example is a unit mass on a unit spring, whose Riccati solution is known in closed form, and a Cayley transform at the end reduces the continuous-time equation to the discrete one so that a single piece of code solves both.

October 07, 2026

Contents

1. The discrete Riccati equation, and the loop that solves it slowly 3
2. The iterate is a graph, and one step is one application of a matrix 4
3. Squaring doubles the horizon 7
4. The powers cannot be formed directly 8
5. Squaring the factorization instead 9
6. The running example 11
7. The continuous equation by the same code 13
8. Where the method is limited, and where it sits 15
9. Sources and further reading 16

Bibliography 17

1. The discrete Riccati equation, and the loop that solves it slowly

A discrete-time plant $x_{k+1} = Ax_k + Bu_k$ with an infinite-horizon cost $\sum_k \left(x_k^\top Q x_k + u_k^\top R u_k\right)$ is driven optimally by the feedback $u_k = -Kx_k$. The optimal cost from a state x is the quadratic form $x^\top X x$, and the cost matrix X solves the **discrete-time algebraic Riccati equation** (DARE)

$$X = Q + A^\top X(I + GX)^{-1}A, \quad G = BR^{-1}B^\top, \quad Q = Q^\top \geq 0, \quad G = G^\top \geq 0. \tag{1}$$

Among its solutions, the one a controller needs is the **stabilizing** solution $X_+ \geq 0$, whose closed loop $K = \left(I + GX_+\right)^{-1}A$ has every eigenvalue inside the unit circle. The equation is nonlinear and, written like this, gives no hint where to start; so the first thing to notice is that it has a direct meaning that suggests a loop.

Let X_j be the optimal cost when the controller has only j steps left to run. Extending a j-step horizon by one step gives the recursion

$$X_{j+1} = Q + A^\top X_j \left(I + GX_j\right)^{-1} A, \quad X_0 = 0, \tag{2}$$

and the finite-horizon costs increase to the infinite-horizon answer: $0 = X_0 \leq X_1 \leq X_2 \leq \ldots \to X_+$. This is the **value iteration**. It is correct and easy, and it is also slow exactly when the problem is interesting. Each step multiplies the error by about $\sigma = \rho\Big(\left(I + GX_+\right)^{-1}A\Big)$, the spectral radius of the closed loop, so a loop close to the unit circle – σ close to 1 – makes the iteration crawl.

The running problem. Throughout, the equation is solved on one plant: a unit mass on a unit spring, actuated by a force – the phase-space plant of the symplectic-matrix note (`Control/symplectic-matrix/symplectic-matrix.pdf`, page 6). In discrete form the data are

$$A = \begin{pmatrix} 0 & -2/3 \\ 1 & 1/3 \end{pmatrix}, \quad G = \frac{1}{3}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 1 & 0 \\ 0 & 2/3 \end{pmatrix}, \tag{3}$$

with $R = 1$ and $B = \left(1/\sqrt{3}\right)\binom{1}{1}$, so that $BR^{-1}B^\top = G$. Its stabilizing solution is

$$X_+ = \begin{pmatrix} 1.912290315 & 0.414213562 \\ 0.414213562 & 1.352193449 \end{pmatrix}, \quad \sigma = 0.53101006. \tag{4}$$

These numbers are not arbitrary: they are the **Cayley image** of the same plant in continuous time, a reduction derived in Section 7, so X_+ there is the continuous Riccati solution and Equation 4 is its exact closed form. Having one plant behind both equations keeps a single set of numbers running through the whole note. On Equation 3 the value iteration needs 27 steps to reach machine precision; the doubling algorithm needs six.

2. The iterate is a graph, and one step is one application of a matrix

The horizon recursion Equation 2 looks nonlinear, but it is **linear in a subspace** once the right object is used. Stack the identity under the current cost,

$$\begin{pmatrix} I_n \\ X_j \end{pmatrix} \in \mathbb{R}^{2n \times n}, \tag{5}$$

and call its span the **graph** of X_j. Writing $K_j = (I + GX_j)^{-1} A$, the recursion says two things at once: $A = (I + GX_j)K_j$ and $X_{j+1} - Q = A^\top X_j K_j$. Stacked, they become one block-matrix equation,

$$\begin{pmatrix} A & 0 \\ -Q & I \end{pmatrix} \begin{pmatrix} I \\ X_{j+1} \end{pmatrix} = \begin{pmatrix} I & G \\ 0 & A^\top \end{pmatrix} \begin{pmatrix} I \\ X_j \end{pmatrix} K_j. \tag{6}$$

The first block row is the definition of K_j rearranged; the second is Equation 2. So Equation 6 carries no new information – but it lets us isolate the matrix

$$\mathcal{S} = \begin{pmatrix} I & G \\ 0 & A^\top \end{pmatrix}^{-1} \begin{pmatrix} A & 0 \\ -Q & I \end{pmatrix}, \tag{7}$$

which carries the graph of X_j to the graph of X_{j+1}:

$$\begin{pmatrix} I \\ X_{j+1} \end{pmatrix} = \mathcal{S}^{-1} \begin{pmatrix} I \\ X_j \end{pmatrix} K_j. \tag{8}$$

Right-multiplying by the invertible K_j does not change a set of columns, so $\operatorname{span}\begin{pmatrix} I \\ X_{j+1} \end{pmatrix} = \mathcal{S}^{-1} \operatorname{span}\begin{pmatrix} I \\ X_j \end{pmatrix}$. **One step of the value iteration is one application of $\mathcal{S}^{-1}$ to the graph of the current cost.** This is the representation change the rest of the note rests on: the horizon recursion is replaced by a fixed linear map acting on subspaces.

To see that action, take the smallest case. Drop to one state, $n = 1$, and use the scalar problem $A = 1.2$, $G = 1$, $Q = 0.1$. Here the graph of X_j is a single line through the origin, the line spanned by $(1, X_j)$, and $\mathcal{S}^{-1}$ moves that line toward one fixed line. Figure 1 draws the successive lines for $j = 0, 1, \ldots, 8$, together with the two invariant lines of $\mathcal{S}$. The lines rotate onto the line that solves the equation and stop there; the matrix case is the same dynamics acting on n-dimensional graphs instead of lines, and the reader should keep this picture for the rest of the argument.

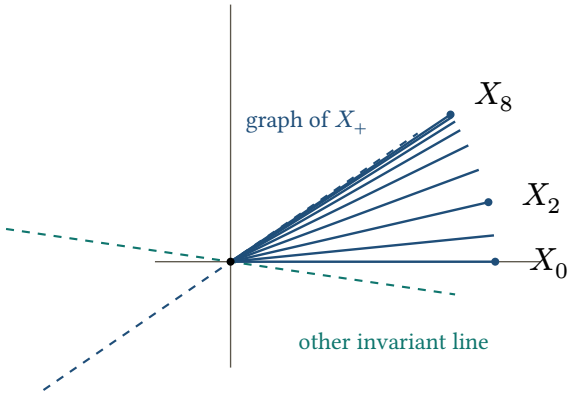


Figure 1: The value iteration as a sequence of subspaces, in the scalar case $A = 1.2$, $G = 1$, $Q = 0.1$. Each solid navy ray is the graph of one X_j; the operation from one ray to the next is "apply $\mathcal{S}^{-1}$, then rescale so the top entry is 1". The dashed navy line is the graph of X_+ – an invariant line of $\mathcal{S}$ – and the rays rotate onto it, crowding together as they arrive, which is the linear convergence of Equation 2. The dashed teal line is $\mathcal{S}$'s other invariant line, which the iteration pushes away. The scalar case is used here only because a graph can be drawn as a line; the non-commuting order of the matrix updates is what the 2×2 running example tests.

The fixed map $\mathcal{S}$ is not arbitrary: with $J = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}$ it satisfies $\mathcal{S}^\top J \mathcal{S} = J$, which makes it **symplectic**.

The closed loop is $\mathcal{S}$ on the stable subspace. At the solution the one-step relation stops moving. Put $X_j = X_{j+1} = X_+$ and $K = (I + GX_+)^{-1}A$ in Equation 8 using $A = (I + GX_+)K$ and $X_+ - Q = A^\top X_+ K$, the two blocks of Equation 6 assemble into

$$\begin{pmatrix} A & 0 \\ -Q & I \end{pmatrix}\begin{pmatrix} I \\ X_+ \end{pmatrix} = \begin{pmatrix} I & G \\ 0 & A^\top \end{pmatrix}\begin{pmatrix} I \\ X_+ \end{pmatrix}K, \quad \text{so} \quad \mathcal{S}\begin{pmatrix} I \\ X_+ \end{pmatrix} = \begin{pmatrix} I \\ X_+ \end{pmatrix}K. \tag{9}$$

The graph of X_+ is therefore an invariant subspace of $\mathcal{S}$, and on it $\mathcal{S}$ acts as the matrix K. That matrix is the closed-loop state matrix, not a new object. The optimal gain is $K_{\text{lqr}} = (R + B^\top X_+ B)^{-1} B^\top X_+ A$, and the push-through identity $B(R + B^\top X_+ B)^{-1} B^\top X_+ = (I + GX_+)^{-1} GX_+$ turns $A - BK_{\text{lqr}}$ into exactly K:

$$A - BK_{\text{lqr}} = \left[I - (I + GX_+)^{-1} GX_+\right] A = (I + GX_+)^{-1} A = K. \tag{10}$$

So Equation 9 reads: **the closed-loop state matrix is $\mathcal{S}$ restricted to the graph of X_+**, written in the basis $\begin{pmatrix} I \\ X_+ \end{pmatrix}$.

The two spectra. Two facts now fix the eigenvalues. Equation 9 makes $\sigma(K)$ a subset of $\sigma(\mathcal{S})$. Symplecticity forces $\sigma(\mathcal{S})$ to be closed under $\lambda \mapsto \frac{1}{\lambda}$ and conjugation: if λ is an eigenvalue then so are $\frac{1}{\lambda}$, $\overline{\lambda}$, and $\frac{1}{\overline{\lambda}}$. Since the closed loop is stable, $\sigma(K)$ lies inside the unit circle, so it is exactly the inside half of $\sigma(\mathcal{S})$, and the outside half is its reciprocal conjugate:

$$\sigma(\mathcal{S}) = \sigma(K) \cup \left\{ \frac{1}{\overline{\lambda}} : \lambda \in \sigma(K) \right\}. \tag{11}$$

For Equation 3 the four eigenvalues are

$$\begin{aligned} &0.10998 \pm 0.51950i \quad (\text{modulus } 0.53101), \\ \text{and} \quad &0.39002 \pm 1.84237i \quad (\text{modulus } 1.88320). \end{aligned} \tag{12}$$

and the inside pair is the closed loop, $\sigma(K) = \{0.10998 \pm 0.51950i\}$, with $1/0.53101 = 1.88320$ recovering the outside pair. That is why the error shrinks by about 0.531 per value-iteration step, and why the graphs slide onto the **stable** subspace. One convention to keep straight: $\begin{pmatrix} I \\ X_+ \end{pmatrix}$ is a basis of the invariant subspace, not a basis of eigenvectors, and it is not orthogonal; the eigenvalues agree because K represents the action of $\mathcal{S}$ on that subspace in this basis, and it is the subspace, not the basis, that the iteration approaches.

3. Squaring doubles the horizon

Apply Equation 8 twice: the graph of X_j becomes the graph of X_{j+1}, then the graph of X_{j+2}. So $\mathcal{S}^{-2}$ moves the horizon by two, and applying the argument k times gives the whole idea:

$$\operatorname{span}\begin{pmatrix} I \\ X_{2^k} \end{pmatrix} = \mathcal{S}^{-2^k} \operatorname{span}\begin{pmatrix} I \\ 0 \end{pmatrix}. \tag{13}$$

Each squaring doubles the number of value-iteration steps that have been taken. The answer is identical – Equation 13 is an identity, not an approximation – but it is reached in k squarings instead of 2^k steps, and the number of **operations** is what matters. The obstacle is that the squaring has to be done in the right representation, and the obvious one fails. Figure 2 shows why the squaring works at all: on the spectrum, squaring raises each inside eigenvalue to a higher power, so it races to 0, while each outside eigenvalue races to ∞. That is the separation the value iteration performs one shadow of per step, now performed 2^k times over.

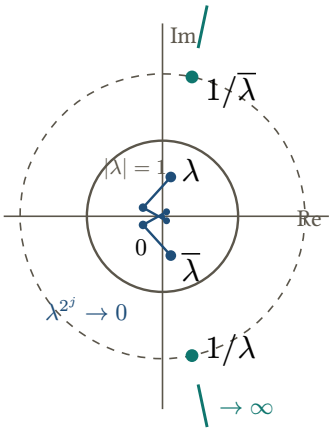


Figure 2: The spectrum of $\mathcal{S}$ under squaring. The solid circle is $|\lambda| = 1$; the dashed circle is $|\lambda| = 1/0.531 = 1.883$. The four eigenvalues are $\lambda, \overline{\lambda}$ inside (navy) and $1/\lambda, 1/\overline{\lambda}$ outside (teal). The small navy dots are successive squares $\lambda, \lambda^2, \lambda^4, \lambda^8$: squaring walks the inside eigenvalue to the origin at a rate that doubles every step. The teal arrows are the mirror effect on the outside pair, which runs to infinity. This is the mechanism the value iteration performs one step at a time and doubling performs 2^k steps at a time.

4. The powers cannot be formed directly

The most direct reading of Equation 13 is to build $\mathcal{S}$, square it k times, and read X_{2^k} off the result. That reading is numerically hopeless, and Figure 2 shows why: the inside eigenvalues are raised to 0.531^{2^k} and vanish, the outside ones to 1.883^{2^k} and explode. After a few squarings the matrix has entries spanning hundreds of orders of magnitude, and the small entries that carry X_+ have been rounded away. Table 1 measures this on Equation 3: the explicit power is fine for two squarings, has lost six digits by the fifth, and is numerically singular by the sixth. Since the arithmetic is in double precision, more squaring does not help: the accurate information is destroyed **before** the iteration has converged.

k	EXPLICIT POWER ERROR	FACTORED-FORM ERROR
0	4.119×10^{-16}	3.413×10^{-16}
1	1.052×10^{-15}	8.546×10^{-16}
2	1.141×10^{-14}	7.756×10^{-15}
3	2.903×10^{-13}	2.134×10^{-13}
4	7.403×10^{-11}	5.847×10^{-11}
5	3.350×10^{-6}	2.758×10^{-6}
6	$4.506 \times 10^{+3}$	(matrix singular)

Table 1: Forming $\mathcal{S}^{-2^k}$ explicitly against keeping it factored, on Equation 3. Column 2 is the error of the explicit power; column 3 is the error of the iterate extracted from it. At $k = 6$ the explicit power has lost the answer entirely. Printed by `scripts/verify_sda.m`.

5. Squaring the factorization instead

The repair is to never form the power, and to square the **representation** of $\mathcal{S}^{-1}$ instead. The representation to square is the block form $\mathcal{S}$ already has. For every k there are matrices A_k, G_k, Q_k with

$$\mathcal{S}^{-2^k} = \begin{pmatrix} A_k & 0 \\ -Q_k & I \end{pmatrix}^{-1} \begin{pmatrix} I & G_k \\ 0 & A_k^\top \end{pmatrix}, \quad A_0 = A, \quad G_0 = G, \quad Q_0 = Q, \tag{14}$$

and for $k = 0$ this is exactly Equation 7 with $\mathcal{S}^{-1}$ in place of $\mathcal{S}$. Two facts make Equation 14 do all the work. First, applying it to the starting vector recovers the iterate:

$$\mathcal{S}^{-2^k} \begin{pmatrix} I \\ 0 \end{pmatrix} = \begin{pmatrix} A_k^{-1} \\ Q_k A_k^{-1} \end{pmatrix} = \begin{pmatrix} I \\ Q_k \end{pmatrix} A_k^{-1}, \quad \text{so} \quad Q_k = X_{2^k}. \tag{15}$$

The second block row is the answer, written straight off the factorization. Second, the block form is closed under squaring, so squaring Equation 14 again produces the same shape with new data $A_{k+1}, G_{k+1}, Q_{k+1}$ from the old. Once those three updates are known, the algorithm is complete, and it never touches a badly conditioned power.

Deriving the updates. A symplectic matrix W of the shape on the right of Equation 14 has its standard-symplectic-form data recoverable from its blocks. Writing $W = \left(\begin{smallmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{smallmatrix}\right)$ and expanding $\left(\begin{smallmatrix} A' & 0 \\ -Q' & I \end{smallmatrix}\right)^{-1} \left(\begin{smallmatrix} I & G' \\ 0 & A'^\top \end{smallmatrix}\right)$ gives $W_{11} = A'^{-1}$, $W_{12} = A'^{-1}G'$, $W_{21} = Q'A'^{-1}$, hence

$$A' = W_{11}^{-1}, \quad G' = W_{11}^{-1}W_{12}, \quad Q' = W_{21}W_{11}^{-1}. \tag{16}$$

Now square Equation 14. With $M_k = \left(\begin{smallmatrix} A_k & 0 \\ -Q_k & I \end{smallmatrix}\right)$ and $N_k = \left(\begin{smallmatrix} I & G_k \\ 0 & A_k^\top \end{smallmatrix}\right)$, the square is $M_k^{-1}N_kM_k^{-1}N_k$, and the middle factor is a single product, because $M_k^{-1} = \left(\begin{smallmatrix} A_k^{-1} & 0 \\ Q_kA_k^{-1} & I \end{smallmatrix}\right)$:

$$N_kM_k^{-1} = \begin{pmatrix} (I + G_kQ_k)A_k^{-1} & G_k \\ A_k^\top Q_k A_k^{-1} & A_k^\top \end{pmatrix}. \tag{17}$$

Multiplying by M_k^{-1} on the left and N_k on the right and reading off the three blocks of Equation 16 gives the updates

$$\begin{aligned} A_{k+1} &= A_k(I + G_kQ_k)^{-1}A_k, \\ G_{k+1} &= G_k + A_kG_k(I + Q_kG_k)^{-1}A_k^\top, \\ Q_{k+1} &= Q_k + A_k^\top(I + Q_kG_k)^{-1}Q_kA_k. \end{aligned} \tag{18}$$

The two forms in the middle line agree because of the identity $(I + G_kQ_k)^{-1}G_k = G_k(I + Q_kG_k)^{-1}$, which is just $(I + G_kQ_k)G_k = G_k(I + Q_kG_k)$ inverted, and its transpose. For instance the second term of G_{k+1} is $A_k(I + G_kQ_k)^{-1}G_kA_k^\top = A_kG_k(I + Q_kG_k)^{-1}A_k^\top$. The result Equation 18 is the **structure-preserving doubling algorithm** (SDA): three coupled matrix identities and one matrix inverse per step, with no squaring of an ill-conditioned power. Each $\mathcal{S}^{-2^k}$ is symplectic by construction, so A_k, G_k, Q_k stay symmetric in the pattern the equation demands, and Q_k stays symmetric positive semidefinite at every step [1], [2].

Try it: Compute the first SDA update by hand for a scalar problem with $A = a$, $G = g$, $Q = q$, and compare it with running one value-iteration step twice. What does the formula give for A_1?

With $A_k = a$, $G_k = g$, $Q_k = q$, Equation 18 gives $A_1 = a(1+gq)^{-1}a = a^2/(1+gq)$, $G_1 = g + a^2 g/(1+qg)$, and $Q_1 = q + a^2 q/(1+qg)$. Running Equation 2 twice from $X_0 = 0$ gives $X_1 = q$ and $X_2 = q + a^2 q/(1+gq)$, which is Q_1: the update reproduces two value steps in one, as Equation 15 promises.

6. The running example

On Equation 3 the iterates do what the argument promises. Table 2 lists the first few. Its fifth column is the difference between Q_k and the value iteration's own 2^k-th iterate: it is machine precision at every k, so Equation 15 is an identity, not an approximation, even though only k squarings were taken. The sixth column shows both methods arriving: the doubling iterates reach X_+ at $k = 5$ and the algorithm stops after six squarings, while the value iteration needs 27 steps. Figure 3 plots the two against the work they cost. The value iteration is a straight line on this scale, because each step multiplies the error by the same 0.531; the doubling iterates fall off a cliff, because each step squares what the previous one achieved.

k	Q_k (1,1)	Q_k (1,2)	Q_k (2,2)	$\lVert Q_k - X_{2^k} \rVert$	$\lVert Q_k - X_+ \rVert$
0	1.000000	0.000000	0.666667	–	1.283
1	1.571429	0.285714	1.142857	5.6×10^{-17}	4.394×10^{-1}
2	1.880000	0.400000	1.333333	2.8×10^{-16}	4.245×10^{-2}
3	1.912094	0.414106	1.352050	3.2×10^{-16}	2.865×10^{-4}
4	1.912290	0.414214	1.352193	3.2×10^{-16}	9.621×10^{-9}
5	1.912290	0.414214	1.352193	2.4×10^{-16}	5.027×10^{-16}

Table 2: The doubling iterates on Equation 3. Each row is one squaring, "work" column 1. Column 5 measures exactness: Q_k against the value iteration's own 2^k-th iterate, which is roundoff at every k. Column 6 measures convergence: the same Q_k against X_+. Steps are counted by the common rule "the iterate stops changing by more than 10^{-14} relative to its norm". Printed by `scripts/verify_sda.m`.

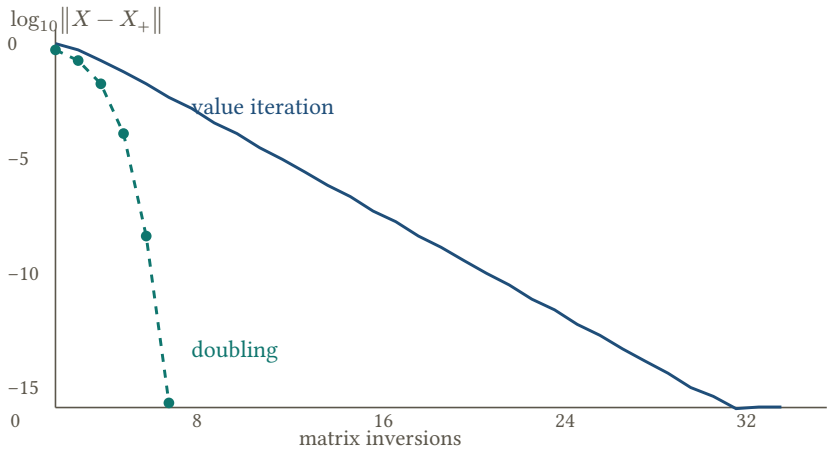


Figure 3: Error against the work, on Equation 3. The horizontal axis counts matrix inversions – one per value-iteration step, one per squaring. The navy curve is the value iteration; the teal markers are the doubling iterates, which are the **same** errors reached at work k instead of 2^k. The value iteration is a straight line of slope $\log_{10} 0.531$; the doubling curve steepens because each step squares the previous error. On this problem the value iteration needs 27 inversions to reach X_+ and doubling needs six.

Why the gap widens. The advantage of doubling is not that it saves a constant factor; it is that the two methods pay different prices for the same accuracy. Table 3 repeats Equation 3 with the state penalty shrunk to $Q = \varepsilon I$, which pulls the closed loop toward the unit circle. As $\sigma \to 1$ the value iteration's line in Figure 3 flattens and its step count grows without bound – it is a fixed-step method chasing a slow mode – while the doubling count grows only like $\log_2$ of that, because each squaring doubles the horizon. The sidecar demo `Control/sda-riccati/sda-riccati-demo.html` lets you drag σ and watch the two curves separate directly.

ε	σ	VALUE ITERATION STEPS	DOUBLING STEPS
10^{-1}	0.802133	73	8
10^{-2}	0.931827	209	9
10^{-4}	0.992954	1752	13
10^{-6}	0.999293	14252	16
10^{-8}	0.999929	109945	19

Table 3: The same plant with $Q = \varepsilon I$ and $R = 1$, reduced to discrete form by the Cayley transform of Section 7 with $\tau = 1$. As the closed-loop radius σ approaches 1, the value iteration's step count explodes while the doubling count grows like its logarithm. Steps use the same rule as Table 2, and the exact solution agrees with the continuous closed form to at worst 1.8×10^{-13}. Printed by `scripts/verify_sda.m`.

7. The continuous equation by the same code

Everything so far was discrete. The same machinery solves the continuous-time Riccati equation, and the bridge is a **Cayley transform**; this is the reduction the CARE version of SDA is built on [2]. The continuous-time equation for the plant $\dot{x} = A_c x + B_c u$ with cost $\int_0^\infty (x^\top Q_c x + u^\top R_c u)\,\mathrm{d}t$ is

$$A_c^\top P + P A_c - P G_c P + Q_c = 0, \quad G_c = B_c R_c^{-1} B_c^\top, \tag{19}$$

whose stabilizing solution P has the closed loop $A_c - G_c P$ in the left half-plane. Its stable subspace is the one belonging to the left-half-plane eigenvalues of the Hamiltonian matrix $\mathcal{H} = \left(\begin{smallmatrix} A_c & -G_c \\ -Q_c & -A_c^\top \end{smallmatrix}\right)$, and it is the graph of P – the continuous analogue of Equation 8.

The reduction uses the Möbius map

$$c(\lambda) = (\lambda + \tau)/(\lambda - \tau) \quad (\tau > 0), \tag{20}$$

which sends the left half-plane into the unit disk and the right half-plane outside it. Applied to the Hamiltonian, $c(\mathcal{H}) = (\mathcal{H} - \tau I)^{-1}(\mathcal{H} + \tau I)$ is a symplectic matrix, and because c is a rational function it maps invariant subspaces to invariant subspaces: $\mathcal{H}\left(\begin{smallmatrix} I \\ P \end{smallmatrix}\right) = \left(\begin{smallmatrix} I \\ P \end{smallmatrix}\right)(A_c - G_c P)$ implies $c(\mathcal{H})\left(\begin{smallmatrix} I \\ P \end{smallmatrix}\right) = \left(\begin{smallmatrix} I \\ P \end{smallmatrix}\right) c(A_c - G_c P)$. The map puts the stable eigenvalues inside the circle – Figure 4 – so the graph of P is the stable subspace of $c(\mathcal{H})$. Writing $c(\mathcal{H})$ in the standard symplectic form Equation 14 produces a discrete Riccati problem with the same solutions:

$$\begin{pmatrix} A_d & G_d \\ -Q_d & A_d^\top \end{pmatrix} = \begin{pmatrix} A_c - \tau I & -G_c \\ Q_c & A_c^\top - \tau I \end{pmatrix}^{-1} \begin{pmatrix} A_c + \tau I & -G_c \\ Q_c & A_c^\top + \tau I \end{pmatrix}. \tag{21}$$

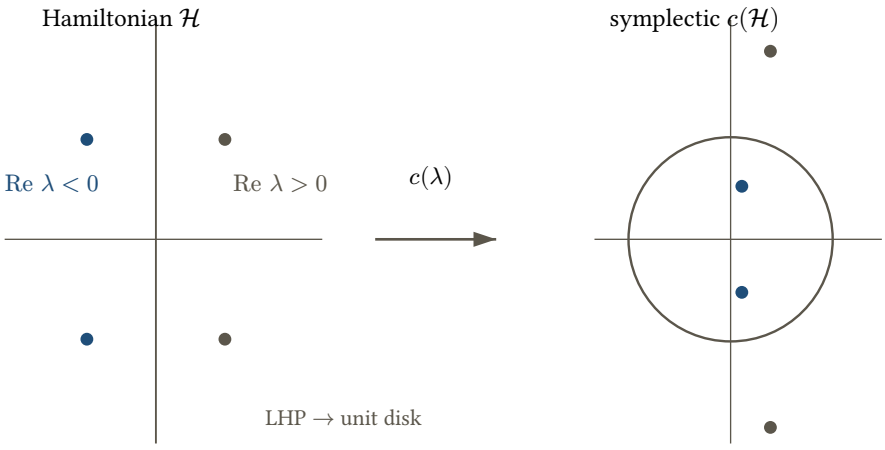


Figure 4: The Cayley transform acting on the spectrum, for the running plant with $\tau = 1$. Left: the Hamiltonian's eigenvalues, a stable pair in the left half-plane (navy) and its mirror pair in the right half-plane (gray). Right: their images under $c(\lambda) = (\lambda + \tau)/(\lambda - \tau)$, which lie inside and outside the unit circle respectively (navy and gray again). The map moves the **stable** set, the left half-plane, into the unit disk, so the stable graph of P on the left is the stable graph of the same P on the right. This is why the discrete data of Equation 3 and the continuous equation Equation 19 have one solution.

For the running plant – unit mass, unit spring, $A_c = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$, $B_c = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, with $Q_c = I$ and $R_c = 1$ – the reduction Equation 21 with $\tau = 1$ returns exactly the discrete data of Equation 3, and its solution is the closed form

$$p_{12} = \sqrt{2} - 1, \quad p_{22} = \sqrt{2p_{12} + 1}, \quad p_{11} = p_{22}(1 + p_{12}), \tag{22}$$

the same expression the companion QZ note solves for (`Control/qz-riccati/qz-riccati.pdf`, page 3). The doubling algorithm on the discrete data returns $P = \begin{pmatrix} 1.912290315 & 0.414213562 \\ 0.414213562 & 1.352193449 \end{pmatrix}$ to 5.0×10^{-16}: the continuous equation has been solved without ever building its Hamiltonian eigensystem. The parameter τ is free, subject only to $A_c - \tau I$ being invertible; different choices give different discrete data and the same solution. On this plant $\tau = 1$ or 2 is fastest (six squarings) and $\tau = 0.25$ or 10 costs two more, with every choice accurate to roundoff.

8. Where the method is limited, and where it sits

The method's one real weakness is inherited, not algorithmic. When $A_c - G_c P$ has an eigenvalue on the imaginary axis, or K an eigenvalue on the unit circle, $\sigma = 1$ and no method can separate the stable subspace from the unstable one; the value iteration stalls outright and SDA degrades from quadratic to linear with rate $1/2$ [3]. The ε rows of Table 3 are the healthy side of that boundary: they show the cost rising smoothly as $\sigma \to 1$, not a cliff.

Two assumptions are worth stating. The factorization Equation 14 and the reduction Equation 21 both need A (and $A_c - \tau I$) invertible; when A is singular the same argument runs on a matrix pencil, which is exactly the deflating-subspace picture of the companion QZ note. And X_+ exists and is unique under stabilizability and detectability, taken as given here.

Finally, SDA is one member of a family. The matrix sign function computes the same stable subspace by iterating $\mathcal{H}_{k+1} = (\mathcal{H}_k + \tau^2 \mathcal{H}_k^{-1})/2$, which is doubling with the Cayley step pushed to the very end; Newton's method converges quadratically too but approaches P from above, while SDA approaches from below; and the ordered Schur and QZ methods of the companion note are direct rather than iterative [3]. SDA is attractive when n is moderate and dense: each step is a handful of $n \times n$ matrix operations, it is short enough to write down, and it needs no eigenvalue decomposition.

> **Try it:** The running example has $\sigma = 0.531$. Roughly how many value-iteration steps does each doubling step save, and why does that ratio keep growing?

Each squaring replaces 2^k value steps by one, and the errors satisfy $e_k \approx C e_{k-1}^2$, so the **ratio** of saved work grows with the accuracy already attained: doubling is quadratic in the number of steps, the value iteration linear. That is why the gap between the two curves in Figure 3 widens as they descend.

9. Sources and further reading

The structure-preserving doubling algorithm and its continuous-time reduction are due to Chu, Fan, Lin and Wang [1], [2], building on Anderson's doubling algorithm for the steady-state discrete Riccati equation [4]. The presentation of both as "squaring variants" of a fixed-point iteration, with the factorization Equation 14 and the convergence caveats, follows the comparative introduction by Poloni [3]. The convergence of the algorithm under weaker hypotheses than stabilizability and detectability, and its behavior when $\sigma = 1$, are in the same reference and in the original papers.

Not consulted, and listed only as pointers: the textbook treatment of discrete Riccati theory in Lancaster and Rodman, **Algebraic Riccati Equations**; the SLICOT and MATLAB `dare` implementations; and the low-rank and decoupled variants for large sparse problems, which are a different regime from the dense problems this note addresses.

Bibliography

[1] E. K.-W. Chu, H.-Y. Fan, W.-W. Lin, and C.-S. Wang, "Structure-Preserving Algorithms for Periodic Discrete-Time Algebraic Riccati Equations," *International Journal of Control*, vol. 77, no. 8, pp. 767–788, 2004, doi: 10.1080/00207170410001714988.

[2] E. K.-W. Chu, H.-Y. Fan, and W.-W. Lin, "A Structure-Preserving Doubling Algorithm for Continuous-Time Algebraic Riccati Equations," *Linear Algebra and its Applications*, vol. 396, pp. 55–80, 2005, doi: 10.1016/j.laa.2004.10.010.

[3] F. Poloni, "Iterative and Doubling Algorithms for Riccati-Type Matrix Equations: A Comparative Introduction," *arXiv preprint*, 2020, [Online]. Available: https://arxiv.org/abs/2005.08903

[4] B. D. O. Anderson, "Second-Order Convergent Algorithms for the Steady-State Riccati Equation," *International Journal of Control*, vol. 28, no. 2, pp. 295–306, 1978, doi: 10.1080/00207177808922455.